

PATHFIND BACKEND DEVELOPER IMPLEMENTATION REPORT

Audit of Implemented vs Unimplemented Features & Database Schema Deliverables

Project:	Pathfind Mentorship Platform (Product Family 2)
Date:	September 11, 2026
Target Role:	Backend Developer (FastAPI + SQLAlchemy + PostgreSQL)
Scope Source:	Figma Canvas (ml9aOY09sY4KW0hEalPFnw) & Project Deliverables Sheet

1. Executive Audit Summary

This report provides a clear breakdown for the Backend Developer detailing what has been implemented so far in the FastAPI backend, what features remain unimplemented, and the exact database models, schemas, and API endpoints that need to be built to complete the backend.

2. What Has Been Implemented (Current State)

The current backend located in backend/ contains foundational user authentication and basic request creation logic. All 9 unit tests in backend/tests currently pass cleanly.

2.1 Implemented Database Models (backend/models.py)

- **User Model:** Stores user primary key (Integer id), unique email, hashed_password, role ('mentee' | 'mentor'), and created_at timestamp.
- **MentorshipRequest Model:** Stores request UUID string id, mentee_id (String), mentor_id (String), request_type Enum, subject, message, status Enum (PENDING, ACCEPTED, DECLINED, COMPLETED), created_at, and updated_at.
- **RequestType Enum:** CV_REVIEW, PORTFOLIO_FEEDBACK, CAREER_PATH_CONVERSATION, INTERVIEW_PREPARATION, ROLE_INDUSTRY_INSIGHT.
- **RequestStatus Enum:** PENDING, ACCEPTED, DECLINED, COMPLETED.

2.2 Implemented API Endpoints (backend/main.py)

- **GET /:** Root welcome status message.
- **GET /health:** API health check route.
- **POST /auth/signup:** User registration (hashes password with bcrypt, stores User).
- **POST /auth/signin:** User authentication (validates password, returns OAuth2 JWT access_token).
- **POST /mentorship-requests:** Creates a new mentorship request (requires JWT bearer auth, validates mentor role).
- **GET /mentorship-requests:** Lists mentorship requests filtered by logged-in user role (mentee vs mentor).
- **GET /mentorship-requests/{request_id}:** Retrieves single request details (validates request recipient/creator).
- **GET /mentorship-request-types:** Returns list of available RequestType enum strings.

3. What Has NOT Been Implemented (Gaps & Missing Features)

A gap analysis comparing the backend against the Figma UI canvas and project scope reveals critical missing features:

- **Mentor Profiles Table & Models:** No MentorProfile model exists to store job_title, company, years_of_experience, bio, expertise_tags, availability, or avatar_url.
- **Mentor Directory Search & Filter API (GET /mentors):** No endpoint exists to search mentors by text query or filter by skill tags (Python, Figma, AWS, etc.) or request types.
- **Single Mentor Detail API (GET /mentors/{id}):** No endpoint exists to fetch a mentor's complete profile for display in the frontend detail modal.
- **Request Accept / Decline Action (PATCH /mentorship-requests/{id}/status):** Mentors have NO route to accept or decline incoming requests or attach a response_message/decline note.
- **Request Attachments (resume_url, portfolio_url, github_url):** MentorshipRequest lacks columns to store links to CVs, portfolios, or code repos submitted by mentees.
- **Organization Email Verification & Account Status:** User model lacks verification_status (PENDING_VERIFICATION, VERIFIED, REJECTED) and domain check for mentor signup (@company.com).
- **Admin Pending Verification Queue (GET /admin/mentors/pending):** No admin endpoint or RBAC check exists to view and approve/reject pending mentor applications.
- **Session Restore API (GET /auth/me):** No route exists to return the current authenticated user's object to restore frontend session state upon page reload.
- **Database Foreign Keys:** mentee_id and mentor_id in MentorshipRequest are plain String columns without ForeignKey('users.id') referential integrity.

4. Step-by-Step Implementation Blueprint for Backend Developer

Step 1: Update Models & Database Schema (backend/models.py)

Refactor backend/models.py to add MentorProfile, VerificationStatus, Foreign Keys, and attachments:

```

class VerificationStatus(str, PyEnum):
    PENDING_VERIFICATION = "pending_verification"
    VERIFIED = "verified"
    REJECTED = "rejected"

class User(Base):
    __tablename__ = "users"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, index=True)
    email: Mapped[str] = mapped_column(String(255), unique=True, index=True, nullable=False)
    hashed_password: Mapped[str] = mapped_column(String(255), nullable=False)
    role: Mapped[str] = mapped_column(String(50), default="mentee", nullable=False)
    verification_status: Mapped[VerificationStatus] = mapped_column(
        Enum(VerificationStatus), default=VerificationStatus.VERIFIED, nullable=False
    )
    profile: Mapped["MentorProfile | None"] = relationship("MentorProfile", back_populates="user", uselist=False)

class MentorProfile(Base):
    __tablename__ = "mentor_profiles"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, index=True)
    user_id: Mapped[int] = mapped_column(Integer, ForeignKey("users.id"), nullable=False, unique=True)
    full_name: Mapped[str] = mapped_column(String(255), nullable=False)
    job_title: Mapped[str] = mapped_column(String(255), nullable=False)
    company: Mapped[str] = mapped_column(String(255), nullable=False)
    years_of_experience: Mapped[int] = mapped_column(Integer, nullable=False)
    bio: Mapped[str] = mapped_column(Text, nullable=False)
    expertise_tags: Mapped[str] = mapped_column(String(500), nullable=False)
    availability: Mapped[str] = mapped_column(String(255), nullable=False)
    avatar_url: Mapped[str | None] = mapped_column(String(500), nullable=True)

class MentorshipRequest(Base):
    __tablename__ = "mentorship_requests"
    id: Mapped[str] = mapped_column(String(36), primary_key=True, default=lambda: str(uuid4()))
    mentee_id: Mapped[int] = mapped_column(Integer, ForeignKey("users.id"), nullable=False, index=True)
    mentor_id: Mapped[int] = mapped_column(Integer, ForeignKey("users.id"), nullable=False, index=True)
    request_type: Mapped[RequestType] = mapped_column(Enum(RequestType), nullable=False, index=True)
    subject: Mapped[str] = mapped_column(String(255), nullable=False)
    message: Mapped[str] = mapped_column(Text, nullable=False)
    resume_url: Mapped[str | None] = mapped_column(String(500), nullable=True)
    portfolio_url: Mapped[str | None] = mapped_column(String(500), nullable=True)
    github_url: Mapped[str | None] = mapped_column(String(500), nullable=True)
    response_message: Mapped[str | None] = mapped_column(Text, nullable=True)
    status: Mapped[RequestStatus] = mapped_column(Enum(RequestStatus), default=RequestStatus.PENDING,
    nullable=False)

```

Step 2: Build New API Endpoints (backend/main.py)

- **GET /auth/me:** Return current authenticated User object + profile data.

- **POST /auth/signup/mentor:** Accept full mentor profile payload & set verification_status = PENDING_VERIFICATION.
- **GET /mentors:** Filter by query, expertise, request_type, verified_only.
- **GET /mentors/{id}:** Return single mentor detail payload.
- **PATCH /mentorship-requests/{id}/status:** Accept status='accepted' | 'declined' | 'completed' + response_message.
- **GET /admin/mentors/pending:** Return mentors where verification_status == PENDING_VERIFICATION.
- **POST /admin/mentors/{id}/approve:** Set verification_status = VERIFIED.